Project Record - part 5 of 8

Utensil Theories Transfer Protocol, 17 to 21 October 2026. This part holds: 5. Every experiment (part B: the seed test, the graph method, the greenhouse ladder, the numbers test).

5. Every experiment (part B: the seed test, the graph method, the greenhouse ladder, the numbers test)

The fair seed test (SEED_TEST_2026-10-20)

Where it came from. On 20 October the owner described their "data seed": a short expression of numbers and symbols, plus fixed grow instructions (add, subtract, multiply, restart or work backwards) that both devices already hold, so only the seed travels. The owner reported "100% reconstruction accuracy and 98.9% data reduction" in Copilot and Gemini chats. They also reported an earlier 99.8% that "required huge files". The AI built an AI-free test that runs on any computer: `SEED_TEST_2026-10-20/seed_test.py`.

What makes it fair:

- **Every byte counts.** Seed and original are both counted as raw bytes, so syntax or encoding cannot hide size.
- **Exact means exact.** The regrown file must match the original byte for byte, checked by SHA-256 fingerprint (a short code that changes if a single bit changes).
- **No peeking.** Each seed is grown by a separate program in an empty folder that holds only the seed. A chat, by contrast, can see the original earlier in the conversation.
- **Grow instructions counted once.** The script plus the method file are shared in advance, and their size is printed.
- **Hard cases.** The test includes random bytes, fresh files the builder never saw, and limits of 60 seconds to plant and 10 seconds to grow.

Results (percent smaller; every method regrew every file exactly, 5 of 5):

Test file	Zip	Graph: fit the shape	Graph: work backwards	Graph: fixed page
Repeating pattern, 120,000 bytes	99.8% (204 bytes)	99.99% (11 bytes)	99.99% (10 bytes)	0%
Short text, 267 bytes	66.3%	64.4%	63.7%	-1.5%
Bible text, 100,000 bytes	70.2%	19.3%	6.2%	0%
Word file, 11,110 bytes	9.9%	1.5%	1.9%	0%
Random bytes, 100,000	0%	0%	0%	0%
All five together	57.7%	42.1%	38.2%	0%

Source: `SEED_TEST_2026-10-20/results_graph_methods.txt`.

The counting limit, in plain words. There are more possible files than short seeds. A 100-byte file can be any of about 10^240 files, but there are only 256 one-byte seeds. So no fixed rules can give every file its own short seed. A method that shrinks some files must leave others the same size or bigger. Random data has no pattern to expect, so it never shrinks, with any method.

The graph method, built three ways

The owner's description (20 October):

- "Use a graph system".
- Points that are only "0" or "1", stacked from the work of solving an expression, make "a page of binary rapidly and it doesn't change".
- "Library style factors called strawberry and orange verse cucumber".

Separate builder agents made the three closest readings:

- **Fit the shape** (`methods/graph_fit.py`). Stretches that a small add/subtract/multiply expression regrows exactly (lines, curves, cycles) are stored as that expression. The rest is stored as it is (the cucumber).

- **Work backwards** (`methods/graph_backwards.py`). Solve back from the data to the rule's unknowns with real algebra, and keep a rule only where it costs less than the bytes it regrows.
- **Find it on a fixed page** (`methods/graph_locate.py`). One fixed expression grows an unchanging page of 0/1 points. The seed lists where each 16-bit piece of the file sits on that page. Its factors use the owner's numbers, for example 17 x 32 x 16 x 34 x 47 = 13,908,992.

Strawberry, orange and cucumber are placeholder names. The owner has not yet shared their own factors. If they do, those factors can go in a new method file and face the same test.

The independent checks: all three PASS. Each checker re-ran the builder's numbers and matched them exactly. Round trips (plant, then grow, then compare) all came back exact:

- fit the shape: 881 cases;
- work backwards: 469 cases plus 1,200 random mixed inputs;
- fixed page: 334 of 334.

No method hides zip or any other compressor. On fresh files the checkers chose, the results matched the table:

- fit the shape: a 60,000-byte Bible slice 19.6% (zip 65.1%), Python 23.0% (zip 69.5%);
- work backwards: 4.0% and 13.8%;
- fixed page: 0% on every file.

The checkers had only minor notes. Two docstrings were slightly wrong. Fit the shape slows to about 50 seconds per megabyte. Work backwards misses repeats more than 255 bytes back. A damaged seed is not detected. Full reports: `SEED_TEST_2026-10-20/checks/graph_methods_reports.md`.

What it means.

- **The seed works where the data is a pattern.** A repeating pattern shrinks to 10 or 11 bytes, which beats zip. That is where the owner's 98.9% shows up.
- **Everyday text is not shaped like lines or cycles.** It stays mostly cucumber, and zip does better.
- **The fixed page never saves space.** A 16-bit piece can be any of 65,536 patterns, and naming one of 65,536 spots takes 16 bits, the same size as the piece. The published "pi filesystem" joke fails for the same reason.

Experiment 12: understanding as compression, the greenhouse ladder (GROW_UNDERSTANDING_2026-10-20)

The owner's question (20 October): "can I grow understanding and compression?" The owner told it as a story: find the soil, build a greenhouse, set the humidity and the sun's rotation, check the air and the bugs, then plant the seed.

The idea. Both sides share a **greenhouse**, knowledge both already hold. The seed carries only what the greenhouse cannot predict. Every word is a **coin toss**. A greenhouse that understands tilts the coin toward what really comes next, so a likely word costs a fraction of a bit. The owner's day-1 phrase, "shift the coin toss towards the desired outcome", fits this unit well. The technical name is arithmetic coding: seed size equals the model's surprise, so understanding and compression become one number.

How it was run. The script `experiments/understanding_compression.py` ran for 23.9 minutes on 20 October.

- **The text:** 623 held-out Bible verses (every 50th verse, never trained on), 83,757 characters.
- **The coder:** an exact whole-number arithmetic coder.
- **The greenhouses:** built from training verses only, and not counted in any seed. The tiny model is about 13.8 MB, so it pays off only when reused for many messages.

The ladder (same text; every seed regrown exactly):

Greenhouse (more understanding further down)	Seed	Smaller than raw
None: raw text	83,757 bytes	0%
Shared word list, every word equally likely	31,262	62.7%
Zip	28,565	65.9%

Greenhouse (more understanding further down)	Seed	Smaller than raw
Zip with a shared 32 KB library	27,625	67.0%
xz (a strong standard compressor)	26,420	68.5%
Word frequencies	19,773	76.4%
Word pairs	14,864	82.3%
Tiny Bible model (3 runs)	12,153-12,252	85.4%
Tiny model + shifting word (markers paid for)	12,411-12,524	85.1%
Plain label, carry3 or scrambled marker	12,359-12,471	85.1-85.2%

The tiny model needed less than half of xz's seed. Its held-out loss had predicted about 11.7 KB; the real coder measured 12.2 KB.

More context helps (the plain model, seeing only the last k words):

Words the model can see	1	2	4	8	16	64
Seed (bytes)	15,111	13,800	13,020	12,508	12,278	12,252

The shifting word: NOT CONFIRMED (pre-registered). Pre-registered means the pass/fail rule was written before the run. The question was whether the shifting-word model regrows the verses with a smaller seed than the plain model (one-sided, $p < 0.05$).

- **It went the other way:** +3.45 bits per verse (+269 bytes in all), larger on 496 of 623 verses, $p = 1.0$.
- **Markers free:** even with the markers given for free, the words cost +0.44 bits per verse more.
- **Against the scrambled control:** a tie on words, +0.04 bits per verse, two-sided $p = 0.69$.

The shifting word's earlier gains were in judging right and wrong, not in predicting the next word.

The palm seed in Antarctica, measured. The owner asked what happens when a palm seed's instructions are followed on a different "OS". The checker decoded the 2-thread seed with 1 thread (one processor lane instead of two). It regrew 547 of 623 verses, then lost step and garbled the rest. A greenhouse made of decimal numbers must be rebuilt identically, down to software versions and thread count, or the seed dies partway. The whole-number rungs are portable.

The independent check: PASS. The checker:

- rebuilt the text and the split, and found no leakage (test text sneaking into training);
- recomputed the simple rungs;
- regrew 50 verses with a coder of a different design;
- reproduced every fingerprint and statistic.

Corrections, all recorded:

- "24 of 24 decodes" is really 23 distinct decodes.
- The printed model size (3,439,680 numbers) is the shifting model's; the plain model has 3,438,720.
- The pre-registration was written into the script before the run, but not committed to git first.

Report: `GROW_UNDERSTANDING_2026-10-20/reports/ladder_build_and_check.md`.

The library result: 98.6% against the owner's 98.9%. With the whole Bible on the shared shelf, naming one verse costs log2(31,102) = 14.9 bits, about 1.9 bytes, against 132.5 bytes for an average verse. That is 98.6% smaller and exact. It fits "it required huge files": the library is the huge file. The records call this **a likely explanation, not a proven one**. It only works for what is already on the shelf. Understanding is what works on new text.

The mirror in the seed expression. In plain characters (* for the owner's multiplication sign, DIV for their division sign) the expression is `[+2*y(27/5.2+t=y/?) DIV (?\y=t+2.5\72)yx2]`. Reverse the left half and mirror each sign (/ to \, (to), the multiplication sign to the letter x), and you get the right half exactly. Only the leading + has no partner. This was checked by computer. Reading it as "growing is planting run in reverse" is an interpretation.

The 15 trained models and the Bible text were committed on 20 October (commit 33a893d), so these seeds can be regrown later.

The numbers test (NUMBERS_TEST_2026-10-21)

The request (20 October, 19:09 UTC): test the numbers "17,32,16,34"=?0=-0=+0=00=["47,"="17+4"]. The owner said they did not know what it meant, only that "the +4 was a way to direct you to a way to try testing the numbers where I feel like they belong."

The reading. Three independent AI readers and a judge settled it before any training. Their write-ups are in `NUMBERS_TEST_2026-10-21/design/interpretations_and_design.md`.

- **Taken as certain:** the order 17, 32, 16, 34, 47, and the slot: the place of 17 in the owner's day-1 rule "W1 = 17 + 4 Remaining".
- **What the slot is:** the **charge** the word gets each time it appears, with the **leftover** from last time carried on top. This recharge was switched off in every earlier experiment, so this is the first test of the owner's full rule.
- **Left untested:** ?0, -0, +0 and 00 are not tested, because no one knows what they mean. Reading "47 = 17+4" as "47 goes in the slot" is a guess, and one version tests it.

The old recharge switch could not test the numbers. It spent too little per verse, so the balance stuck at the cap (+/-51). Swapping in any of the owner's numbers gave 100.0% the same marked text.

The rule, per verse:

1. The word is spelled from the Remaining (R), using the same five spellings as before, from anoksha (heavy wrong) through velanok (unresolved) to ruvelan (heavy right).
2. If a direction exists, the next charge is added in that direction: W = R + charge.
3. A clear right or wrong verse sets R to +17 or -17 and restarts the list.
4. Otherwise the verse's valued words add their values to W, and W shrinks by 14.1% per valued word (the "drain"). The result is the new R, capped at +/-51.

The drain was the AI's choice, not the owner's. It is 0.141073 per valued word, worked out from the 10.9043 valued words in an average verse. It is set so the plain rule settles at exactly the owner's numbers: W = 17 + 4 = 21, each verse uses 17, and 4 remain. The dry run confirmed "W = 21.000, Remaining = 4.000". This is what stops the word sticking at the top.

The six versions (3 training runs each = 18 tiny models):

Version	Charge each time	Why
num_charge17	17	The owner's plain recharge rule
num_seq	17 at a clear choice, then 32, 16, 34, 47, 17 ...	The owner's numbers, in order
num_seqrev	17, then 47, 34, 16, 32 ...	Does the order matter?
num_const29	29.2 (the average of the five)	Sequence, or just a bigger charge?
num_charge47	47	47 in the place of "17+4"
num_drain0	0	Stops the AI's drain from taking credit

The comparators are earlier models, reused unchanged: the current shifting word (the main one), the plain Bible model, and label, carry3 and scrambled (description only).

Measures:

- **M1, judging (main):** 100 new story pairs, one right and one wrong.
- **M2, long stories:** the act scrolls out of the model's 64-word view.
- **M3, exact seed size:** bits to regrow the 623 unseen verses exactly. Run 0 must really regrow them, or M3 is void.

Pass/fail rules, written before training (top of `experiments/numbers_test.py`):

- **Tests:** one-sided: sign tests (counting wins and losses pair by pair) for M1 and M2, and a 100,000-flip permutation test (reshuffling to see how often luck matches the real gap) for M3. Holm correction at 0.05 within each group, a guard against lucky wins across many comparisons.
- **Groups:** each version against the shifting word (F1) and against the plain model (F2). The owner's numbers (F3): N1 num_seq beats num_charge17, N2 beats num_seqrev, N3 beats num_const29, and N4 num_charge47 beats num_charge17. Mechanism (R1): num_charge17 beats num_drain0.

- **Verdicts on M1:**
  - "The owner's numbers matter" = N1 passes.
  - "The sequence itself matters" = N1, N2 and N3 pass.
  - "47 helps" = N4 passes.
  - "The recharge rule helps" = num_charge17 passes F1, and is credited to the recharge only if R1 also passes.
- **Other outcomes:** a pass on M2 or M3 alone is a "secondary sign". Anything else is "not shown at this scale", never "disproved".
- **Expectation written in advance:** ties with the shifting word on M1, as every marker method has tied so far.

The dry run (21 October, 02:06:50-02:08:30 UTC, "done in 1.7 minutes" by its own log; a chat reply said 2.6 minutes). It ran every part with 5-step models, seed 0 only, on small subsets, writing only to a scratch folder.

The manipulation check tests, before training, that no spelling is stuck. The rule: at most 5% of verses at the cap, and every spelling on at least 3%. **All six pass:**

Version	Spellings, heavy wrong -> heavy right	At cap	Result
num_charge17	10.8 / 23.1 / 32.0 / 20.1 / 13.9%	0.03%	PASS
num_seq	19.1 / 20.4 / 22.2 / 16.7 / 21.6%	0.27%	PASS
num_seqrev	19.7 / 19.8 / 22.1 / 16.5 / 21.9%	0.30%	PASS
num_const29	19.4 / 20.8 / 21.1 / 16.7 / 22.0%	0.26%	PASS
num_charge47	29.0 / 15.3 / 13.5 / 12.2 / 30.0%	1.23%	PASS
num_drain0	4.4 / 17.7 / 52.8 / 19.8 / 5.3%	0.00%	PASS

- **The judge's simulation** was matched within 0.05 points.
- **Reproduction:** the saved shifting and plain models gave exactly their stored scores ("REPRODUCED EXACTLY", 3 of 3 checks).
- **Exact seeds:** all six new seeds were decoded byte for byte.
- **Scores mean nothing:** the dry run's scores come from 5-step models, which have learned almost nothing. Some M3 lines even print "PASS" for this reason alone. Files: `NUMBERS_TEST_2026-10-21/dry_run/`.

Status at about 02:20 UTC, 21 October, updated at 17:45 UTC: NO RESULTS YET.

- **Review passed (21 Oct):** three of four reviewers said "ready" with no blocking problems; the run-safety checks were done directly; ten small fixes left the frozen design unchanged; a second dry run passed; the full run started at 17:45 UTC.
- **Full run (running):** 18 models, about 4-5 hours, saving and pushing every 10 minutes.
- **Limits:** tiny models, 3 runs and 100 pairs mean only large effects can show. Whatever the result, it will not change how a large AI understands anyone.